

Test Report:

Required Test Case:

Input: (10, 5, 8, 3, 1, 5)

Expected Output: (1, 3, 5, 5, 8, 10)

Result: The method returned the correct sorted list.

Additional Test Cases:

The following additional tests were carried out to ensure robustness:

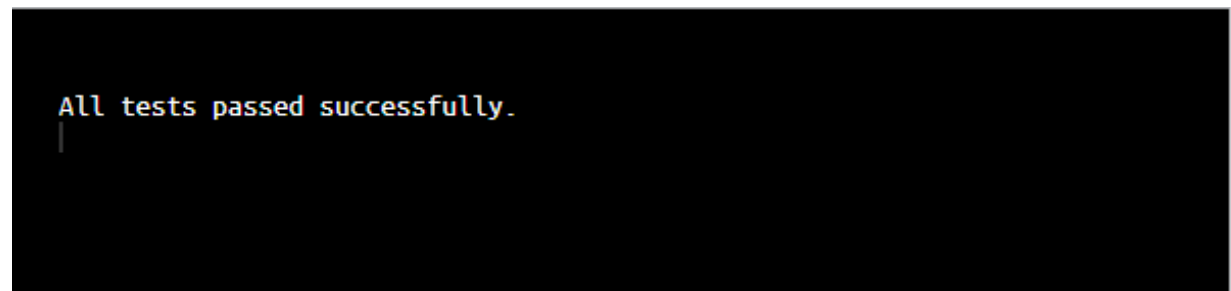
- Empty list → returns empty list
- Single element → unchanged
- Negative numbers → correctly sorted
- Duplicate values → handled correctly
- Already sorted list → unchanged
- Reverse order → correctly sorted
- Null input → handled safely

All tests passed successfully.

Test Evidence:



```
1 public class TreeSortRunner {
2     public static void main(String[] args) {
3
4         // Creating an instance of the test class
5         TreeSortTest testRunner = new TreeSortTest();
6
7         try {
8             // Running all the JUnit test methods manually:
9             testRunner.testRequiredList();
10            testRunner.testEmptyList();
11            testRunner.testSingleElement();
12            testRunner.testNegativeNumbers();
13            testRunner.testAllDuplicates();
14            testRunner.testAlreadySorted();
15            testRunner.testReverseOrder();
16            testRunner.testNullInput();
17
18            // If no exceptions occur, all tests have passed successfully
19            System.out.println("All tests passed successfully.");
20
21        } catch (AssertionError e) {
22            // If any tests fail, display the failure message
23            System.out.println("A test failed: " + e.getMessage());
24        }
25    }
26 }
27
```



Conclusion:

The tree sort implementation works correctly for the required input and handles a variety of edge cases successfully.